MARKETING-PLATFORM-FRONTEND · PACKAGE.JSON

This project uses 44 packages

16 packages ship with the app to production. The other 28 are only for development, building, testing and code quality. Each package's job is explained below in one line.

44
total
Direct packages (their own sub-dependencies are not counted)

16
dependencies
Needed to run the app; included in the production build

28
devDependencies
Used only while developing, building, testing and linting

BREAKDOWN BY GROUP



Core framework 3 · UI and styling 8 · Forms 3 · Data and state 2 · CSS and fonts 6 · TypeScript 4 · Code quality 6 · Testing 8 · Git workflow 4

Ships with the app

dependencies · 16

Core framework 3

The app cannot run without these.

`next`
Framework: folder-based routing, server rendering, the build and the production server (v16.4.3).

`react`
Library for building the UI out of components.

`react-dom`
Renders React components into the browser page (DOM).

UI and styling 8

How components look, plus icons, animation, theming and toasts.

`radix-ui`
Accessible building blocks: Dialog, Popover, NavigationMenu, Accordion. Used in the header and the sheet.

`class-variance-authority`
Defines a component's variant classes (size, level, tone) in one place, e.g. `headingVariants`.

`clsx`
Joins class names based on conditions.

`tailwind-merge`
Resolves conflicting Tailwind classes so the later one wins. Used with clsx in `cn()`.

`lucide-react`
Icons.

`motion`
Animations: opening and closing the sheet, auth flow step transitions.

`next-themes`
Switches between light and dark theme and remembers the choice.

`sonner`
Toast notifications ("Saved", errors).

Forms and validation 3

For forms such as login and signup.

`react-hook-form`
Manages form values, errors and submit with few re-renders.

`zod`
Validates data against a schema; the same schema gives you the TypeScript type.

`@hookform/resolvers`
Connects a Zod schema to react-hook-form (`zodResolver`).

Data and state 2

Data from the server and the app's own UI state.

`@tanstack/react-query`
Fetches, caches and refreshes API data (e.g. the header data).

`zustand`
Small global client state, e.g. `ui-store` (whether the menu is open).

Development only

devDependencies · 28

CSS and fonts 6

CSS and font files are generated at build time.

`tailwindcss`
Utility-class CSS framework (v4).

`@tailwindcss/turbopack`
Runs Tailwind inside the Turbopack bundler (`next.config.ts`).

`@fontsource-variable/inter`
Inter, the body text font.

`@fontsource-variable/plus-jakarta-sans`
Plus Jakarta Sans, the heading font.

`@fontsource-variable/noto-sans-devanagari`
Font for Hindi (Devanagari) text.

`@fontsource-variable/jetbrains-mono`
Code / monospace font.

The fonts are devDependencies because `fonts.ts` reads their files at build time and self-hosts them. The packages aren't needed at runtime.

TypeScript 4

Type checking only; nothing reaches the browser.

`typescript`
The TS compiler; run by `npm run typecheck`.

`@types/node`
Types for Node.js APIs.

`@types/react`
Types for React.

`@types/react-dom`
Types for React DOM.

Code quality 6

Keeps code rules and formatting consistent.

`eslint`
Catches bugs and bad patterns in code.

`eslint-config-next`
ESLint rules for Next.js.

`@tanstack/eslint-plugin-query`
Rules for using React Query correctly.

`eslint-config-prettier`
Turns off ESLint formatting rules so they don't clash with Prettier.

`prettier`
Formats code automatically.

`prettier-plugin-tailwindcss`
Sorts Tailwind classes into a consistent order.

Testing 8

Unit tests (Vitest) and browser tests (Playwright).

`vitest`
Unit test runner (`npm test`).

`@vitejs/plugin-react`
Lets Vitest run JSX / React code.

`vite-tsconfig-paths`
Resolves `@/` import paths in tests.

`jsdom`
Simulates a browser DOM inside Node.

`@testing-library/react`
Renders components so you can test them.

`@testing-library/jest-dom`
Extra assertions such as `toBeInTheDocument()`.

`@testing-library/user-event`
Simulates user actions such as clicks and typing.

`@playwright/test`
End-to-end tests in a real browser (`npm run test:e2e`).

Git workflow 4

Automatic checks before every commit.

`husky`
Runs Git hooks (`pre-commit`, `commit-msg`).

`lint-staged`
Runs lint and format only on staged files.

`@commitlint/cli`
Checks the commit message.

`@commitlint/config-conventional`
Enforces the message format: `feat(auth): ...`, `fix: ...`

How packages work together

3 examples

Styling: `cn()`

Variant classes are built, joined, then conflicts are removed.

`class-variance-authority` → `clsx` → `tailwind-merge` → **final className**

Form submit

The user types, Zod validates, and only valid data goes through.

`react-hook-form` → `@hookform/resolvers` → `zod` → **show errors or submit**

`git commit`

A lint error or a badly formatted message stops the commit.

`git commit` → `husky pre-commit` → `lint-staged` → `eslint + prettier` → `commitlint` → **commit done**

Source: the project's `package.json` (Next.js 16.4.3, React 19.3.1). Only direct packages are counted.